

计算机系统基础 作业二

学号：2410668

姓名：吕昊远

一、为什么使用 jbe 指令？

If-else语句举例

```
int get_cont( int *p1, int *p2 )
{
    if ( p1 > p2 )
        return *p2;
    else
        return *p1;
}
```

p1和p2对应实参的存储地址分别为 R[ebp]+8、R[ebp]+12，EBP指向当前栈帧底部，结果存放在EAX。

作业：为何这里是“jbe”指令？

```
movl 8(%ebp), %eax //R[eax] ← M[R[ebp]+8], 即 R[eax]=p1
movl 12(%ebp), %edx //R[edx] ← M[R[ebp]+12], 即 R[edx]=p2
cmpl %edx, %eax    //比较 p1 和 p2, 即根据 p1-p2 结果置标志
jbe .L1            //若 p1 ≤ p2, 则转 L1 处执行
movl (%edx), %eax
jmp .L2            //无条件跳转到 L2 执行
.L1:
    movl (%eax), %eax
.L2
```

1

这里的 p1 和 p2 是指针，存储的是内存地址，在 x86 汇编中，比较跳转指令分为两类，分别是有符号比较：jle、jge、jl、jg（基于符号标志位 SF 和溢出标志位 OF），与无符号比较：jbe、jae、jb、ja（基于进位标志位 CF）。

由于地址本质上是无符号整数，所以编译器必须使用无符号比较指令来比较两个指针的大小。代码中的条件是 if (p1 > p2)，.L1 是走 else 分支（即条件为假时跳转），也就是当 p1 ≤ p2 时跳转。对无符号数来说，“小于等于”对应的指令正是 jbe（jump if below or equal）。如果编译器错误地使用 jle，当指针地址的最高位为 1（即作为有符号数被视为“负数”）时，比较结果会出错。

二、/lib/ld-linux.so.2 中.2 的含义

加载时动态链接

- 程序头表中有一个特殊的段：INTERP
- 其中记录了动态链接器目录及文件名ld-linux.so

作业：/lib/ld-linux.so.2 中.2 的含义

```
Program Headers:
Type      Offset  VirtAddr  PhysAddr  FileSiz MemSiz  Flg Align
PHDR      0x000034 0x08048034 0x08048034 0x00100 0x00100 R E 0x4
INTERP    0x000134 0x08048134 0x08048134 0x00013 0x00013 R  0x1
[Requesting program interpreter: /lib/ld-linux.so.2]
LOAD      0x000000 0x08048000 0x08048000 0x004d4 0x004d4 R E 0x1000
LOAD      0x000f0c 0x08049f0c 0x08049f0c 0x00108 0x00110 RW 0x1000
DYNAMIC   0x000f20 0x08049f20 0x08049f20 0x000d0 0x000d0 RW 0x4
NOTE      0x000148 0x08048148 0x08048148 0x00044 0x00044 R  0x4
GNU_STACK 0x000000 0x00000000 0x00000000 0x00000 0x00000 RW 0x4
GNU_RELRO 0x000f0c 0x08049f0c 0x08049f0c 0x000f4 0x000f4 R  0x1
```

这里的 ".2" 是动态链接器的主版本，Linux 共享库的命名遵循以下格式：

库名 .so . 主版本号 . 次版本号 . 发布号

在这里，/lib/ld-linux.so.2 表示动态链接器 (ld-linux.so) 的第 2 版。主版本号变化代表 ABI（应用二进制接口）不兼容，也就是针对版本 2 编译的程序无法使用其他版本的链接器。实际上，/lib/ld-linux.so.2 通常是一个符号链接，指向真正的实体文件（例如 ld-2.31.so 这样的具体版本），这种带主版本号的软链接机制，可以让多个不同主版本的库同时存在于系统中互不冲突。

三、符号表在编译器的哪里？

链接操作的步骤

- Step 1. 符号解析 (Symbol resolution)
 - 程序中有定义和引用的符号 (包括变量和函数等)
 - void swap() {...} /* 定义符号swap */
 - swap(); /* 引用符号swap */
 - int *xp = &x; /* 定义符号 xp, 引用符号 x */
 - 编译器将定义的符号存放在一个符号表 (symbol table) 中。
 - 符号表是一个结构数组
 - 每个表项包含符号名、长度和位置等信息
 - 链接器将每个符号的引用都与一个确定的符号定义建立关联

作业：符号表在编译器的哪里？

严格来说，符号表并不在编译器里，它存在于编译工具链生成的可重定位目标文件（.o 文件）中。以 ELF 格式为例，符号表位于目标文件的 .symtab 段，它是一个结构数组，每个表项记录了符号名、长度、类型、所属段以及地址（偏移）等信息；而符号名的实际字符串则单独存放在 .strtab 段中，.symtab 表项中只保存指向 .strtab 的偏移量，这样设计可以避免重复存储字符串。

从生成流程来看，编译器在编译过程中收集源程序中定义和引用的符号信息并输出汇编代码，汇编器再把汇编代码翻译成机器码并同时构造出 .symtab 和 .strtab 段写入 .o 文件，最后链接器读取各个 .o 文件的符号表完成符号解析与重定位。

总之，符号表存在于编译过程产生的.o 文件的 .symtab 节中，由汇编器创建，供链接器使用。